

```
In [1]: #23MIC0006
        #Pranav Anand
```

```
In [2]: # Task 1
        # Creating a simple thread using threading module

import threading
import time

start = time.perf_counter()

# Function executed by the thread
def work():
    print("Sleeping for 1 second")
    time.sleep(1)
    print("Finished sleeping")

# Create and start the thread
thread = threading.Thread(target=work)
thread.start()

# Wait for the thread to finish
thread.join()

end = time.perf_counter()

print(f"Finished in {round(end - start, 2)} second(s)")
```

```
Sleeping for 1 second
Finished sleeping
Finished in 1.03 second(s)
```

```
In [3]: # Task 2
        # Creating multiple threads using threading module

import threading
import time

start = time.perf_counter()

# Function executed by each thread
def work():
    print("Sleeping for 1 second")
    time.sleep(1)
    print("Finished sleeping")

threads = []

# Create and start 10 threads
for count in range(10):
    thread = threading.Thread(target=work)
    thread.start()
    threads.append(thread)

# Wait for all threads to complete
for thread in threads:
    thread.join()
```

```
end = time.perf_counter()

print(f"Finished in {round(end - start, 2)} second(s)")
```

```
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Sleeping for 1 second
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished in 1.13 second(s)
```

```
In [4]: # Task 3
        # Creating multiple threads and passing arguments

import threading
import time

start = time.perf_counter()

# Function that accepts the sleeping time
def work(seconds):
    print(f"Sleeping for {seconds} second")
    time.sleep(seconds)
    print("Finished sleeping")

threads = []

# Create 50 threads
for count in range(50):
    thread = threading.Thread(target=work, args=(1,))
    thread.start()
    threads.append(thread)

# Wait for all threads
for thread in threads:
    thread.join()

end = time.perf_counter()

print(f"Finished in {round(end - start, 2)} second(s)")
```

[illegible]

[illegible]

```
In [5]: # Task 4  
# Creating threads using concurrent futures module  
  
import concurrent.futures  
import time  
  
start = time.perf_counter()  
  
# Function executed by the threads  
def work(seconds):  
    print(f"Sleeping for {seconds} seconds")  
    time.sleep(seconds)  
    return f"Finished sleeping for {seconds} seconds"  
  
# Create a thread pool  
with concurrent.futures.ThreadPoolExecutor() as pool:  
  
    # Submit two tasks
```

```

first = pool.submit(work, 1.5)
second = pool.submit(work, 1.5)

# Get the results
print(first.result())
print(second.result())

end = time.perf_counter()

print(f"Finished in {round(end - start, 2)} second(s)")

```

Sleeping for 1.5 seconds
 Sleeping for 1.5 seconds
 Finished sleeping for 1.5 seconds
 Finished sleeping for 1.5 seconds
 Finished in 1.52 second(s)

In [6]:

```

# Task 5
# Creating multiple threads using list comprehension

import concurrent.futures
import time

start = time.perf_counter()

# Function executed by each thread
def work(seconds):
    print(f"Sleeping for {seconds} seconds")
    time.sleep(seconds)
    return "Finished sleeping"

# Create thread pool
with concurrent.futures.ThreadPoolExecutor() as pool:

    # Create multiple tasks using list comprehension
    jobs = [pool.submit(work, 1.5) for i in range(2)]

    # Get completed results
    for job in concurrent.futures.as_completed(jobs):
        print(job.result())

end = time.perf_counter()

print(f"Finished in {round(end - start, 2)} second(s)")

```

Sleeping for 1.5 seconds
 Sleeping for 1.5 seconds
 Finished sleeping
 Finished sleeping
 Finished in 1.51 second(s)

In [7]:

```

# Task 6
# Passing a list of arguments using map

import concurrent.futures
import time

start = time.perf_counter()

# Function that accepts sleeping time
def work(seconds):

```

```

    print(f"Sleeping for {seconds} seconds")
    time.sleep(seconds)
    print("Finished sleeping")

# List of arguments
time_values = [5, 4, 3, 2, 1]

# Create thread pool
with concurrent.futures.ThreadPoolExecutor() as pool:

    # Execute function for each value
    results = pool.map(work, time_values)

    # Display results
    for result in results:
        print(result)

end = time.perf_counter()

print(f"Finished in {round(end - start, 2)} second(s)")

```

```

Sleeping for 5 seconds
Sleeping for 4 seconds
Sleeping for 3 seconds
Sleeping for 2 seconds
Sleeping for 1 seconds
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
Finished sleeping
None
None
None
None
None
Finished in 5.02 second(s)

```

```

In [8]: # Task 7
        # Downloading multiple audio files without using threads

import requests
import time

# List of media URLs
files = [
    "https://dl.prokerala.com/downloads/ringtones/files/dl/mp3/iphone-remix-6802",
    "https://dl.prokerala.com/downloads/ringtones/files/dl/mp3/flute-62874.mp3",
    "https://dl.prokerala.com/downloads/ringtones/files/dl/mp3/god-flute-2922.mp",
    "https://codeskulptor-demos.commondatastorage.googleapis.com/pang/paza-modul",
    "https://commondatastorage.googleapis.com/codeskulptor-assets/sounddogs/soun
]

start = time.perf_counter()

# Download each file one by one
for link in files:

    with requests.get(link, stream=True) as response:

```

```
# Check whether the request was successful
response.raise_for_status()

# Get file name from URL
filename = link.split("/")[-1]

if filename == "":
    filename = "downloaded_file"

# Save the downloaded file
with open(filename, "wb") as file:

    # Download file in small chunks
    for data in response.iter_content(chunk_size=8192):

        if data:
            file.write(data)

    print(f"{filename} downloaded successfully")

end = time.perf_counter()

print(f"Finished in {end - start:.2f} seconds")
```

```
iphone-remix-68028.mp3 downloaded successfully
flute-62874.mp3 downloaded successfully
god-flute-2922.mp3 downloaded successfully
paza-moduless.mp3 downloaded successfully
soundtrack.mp3 downloaded successfully
Finished in 5.27 seconds
```

In []: